

IMPLEMENTATION OF AI SHOOTER CHALLENGE GAME VIA OBJECT-ORIENTED DESIGN AND BFS PATHFINDING IN JAVA

Author Name (戴銘漢)

Department of Computer Science (資訊工程系)

National Penghu University of Science and Technology (澎湖科技大學)

Magong City, Penghu, Taiwan

Email: d12405049@ems.npu.edu.tw

ABSTRACT—This paper presents the practical design and implementation of a 2D airplane shooting game, titled "AI Shooter Challenge," engineered using the Java programming language and standard graphical components. Adhering to strict academic and architectural standards, the gameplay framework segregates operational logic from spatial-temporal graph traversals. Specifically, the enemy aircraft artificial intelligence is powered by a Breadth-First Search (BFS) pathfinding algorithm mapped onto a discrete grid map coordinate matrix, allowing dynamic obstacle avoidance and real-time trajectory tracking toward the player plane. By incorporating core components under standard Swing frameworks, this study encapsulates clean, scalable class design, validates collision management, and establishes a robust scoring-health ecosystem for automated game lifecycle execution.

Keywords—Java Programming, Object-Oriented Design, BFS Pathfinding, Image Processing, Artificial Intelligence.

I. INTRODUCTION

Real-time game system design and automated agent pathfinding represent major domains within modern Computer Science and functional Artificial Intelligence implementations. To bridge theoretical graph structures with reactive visual output, this study outlines the architecture of an interactive 2D desktop shooting simulation. The main objective is to establish an engaging software prototype wherein host and opponent agents interact deterministically inside a constrained spatial matrix.

Unlike conventional game loops that deploy primitive linear vectors for NPC (Non-Player Character) movement patterns, the "AI Shooter Challenge" architecture enforces programmatic cleverness. Adversaries within this paradigm are governed by graph search mechanisms designed to calculate the absolute shortest path on unweighted grid networks. The structural boundaries of this design dictate that every mechanical operation—ranging from keystroke interrupts to bullet intersection computations—be driven by clean object-oriented class instances.

II. METHODOLOGY

The system architecture utilizes Object-Oriented Programming (OOP) paradigms to distribute individual responsibilities across modular classes, preventing monolithic code degradation. The fundamental design relies on the spatial discretization of the coordinate window into an unweighted uniform grid layout, where each node is designated as either crossable or blocked by stationary obstacles (e.g., meteorites or rocks).

The core pathfinding algorithm deployed for opponent tracking is Breadth-First Search (BFS). Given an unweighted grid configuration $G = (V, E)$, the enemy agent acts as the root node s , looking for a destination node v , which corresponds to the instantaneous position of the player airplane. The traversal process handles exploration via a First-In-First-Out (FIFO) queue data structure. The search visits all orthogonal adjacent neighbors (Up, Down, Left, Right) systematically. The cell mapping state can be represented by the following formulation:

$$M(x, y) = \{ 0: \text{Accessible}, 1: \text{Obstacle} \}$$

When the destination node \$v\$ is intersected, the parent-pointer chain is backtracked to isolate the immediate subsequent grid step for execution. This cycle executes periodically inside the rendering thread, assuring responsive interception behaviors.

III. IMPLEMENTATION DETAILS

The system was compiled and tested within the Java Development Kit (JDK) 17+ environment utilizing the native `javax.swing` graphical framework. The development environment avoids third-party dependencies to maximize native cross-platform thread synchronization. Structurally, the package is separated into functional subdirectories as follows:

```
AI Shooter Challenge/
├── src/
│   ├── main/
│   │   ├── GameFrame.java
│   │   └── GamePanel.java
│   ├── player/
│   │   └── Player.java
│   ├── enemy/
│   │   └── Enemy.java
│   ├── bullet/
│   │   └── Bullet.java
│   ├── map/
│   │   └── Map.java
│   ├── util/
│   │   └── Node.java
│   └── resources/
│       └── images/
```

Fig. 1. Object-Oriented Package Directories Structure Diagram.

The discrete graph logic requires a standalone wrapper entity to encapsulate coordinates. Below is the structural implementation snippet of the `Node` utility class used in the internal BFS execution queue:

```
package util;

public class Node {
    public int x, y;
    public Node parent;

    public Node(int x, int y) {
        this.x = x;
        this.y = y;
        this.parent = null;
    }
}
```

IV. EXPERIMENTAL RESULTS

The fully compiled application yields a high-performance desktop window rendered smoothly through standard hardware buffers. The main display outputs key metrics including real-time score tracking, current level configuration, and remaining health parameters (HP) within a clean visual overlay layout. Key operational controller bounds map perfectly across user runtime environments as cataloged below:

Key Binding	Triggered Operational Event Action Description
WASD / Arrow Keys	Modifies directional displacement vectors of the player airplane object.
SPACEBAR	Instantiates a new Player Bullet object initialized at current source coordinates.
R Key	Resets game state flags, recreates player entities, clears vectors, and restarts the loop.
P Key	Toggles the active state of the main execution rendering timer to pause or resume action.

Table I. System Keyboard Controller Mapping Matrix.

During active execution, the AI enemies trace optimal trajectories and successfully circumvent complex rock arrangements. Projectiles fired from the player plane intersect enemy boundary boxes accurately, generating an increase of +100 points per entity neutralized. Conversely, when an enemy successfully impacts the player, the system decrements the health pool by 1 point. If the health parameter arrives at zero ($\$HP = 0\$$), the rendering engine halts and throws a clear "Game Over" message onto the viewport.

V. DISCUSSION

The structural combination of automated graph search loops and real-time Swing render ticks emphasizes several modular performance insights:

- **Encapsulation and Decoupling:** Keeping the `CollisionManager` distinct from raw drawing structures enables isolated enhancements to the hit detection math (e.g., transitioning from basic bounding boxes to advanced circular checks) without corrupting independent classes.
- **Algorithmic Performance:** Because BFS scales linearly relative to the graph size ($\$O(V + E)\$$), running path computations across standard 15×15 grid arrays introduces negligible computation overhead, maintaining stable frames per second.

VI. CONCLUSION

This project successfully establishes a complete Java-based framework demonstrating the integration of graph pathfinding within an interactive real-time visual environment. By mapping architectural guidelines directly into robust classes, the application ensures stability, deterministic behavior, and smooth event handling. Future expansions will target the optimization of pathfinding overhead via A* algorithms and the inclusion of image serialization utilities to load custom graphic sprite sheets dynamically from external storage folders.

REFERENCES

- [1] M. H. Dai, "Implementation of Probability Theory Lecture Notes via Object-Oriented Design in Java," NPU Academic Writing Series, vol. 1, pp. 1-2, April 2026.
- [2] J. Gosling, B. Joy, G. Steele, and G. Bracha, *The Java Language Specification*, Redwood City, CA: Addison-Wesley, 2014.
- [3] N. Otsu, "A Threshold Selection Method from Gray-Level Histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62-66, 1979.
- [4] X. R. Huang, "Lecture 8: Shooter Challenge and Graph Traversals," Department of Computer Science, National Penghu University of Science and Technology, 2026.